



University of Sri Jayewardenepura

ICT 1062 Object Oriented Programming

ICT 1051 Database Systems & Administration

First Year – Second Semester Individual Final Assignment - 2025

Name: R. K. D. A. Kasundi

Index number: AS20240980

Acknowledgment

I would like to express my sincere gratitude to the lecturer and instructors of ICT 1062 – Object-Oriented Programming for their valuable guidance, encouragement, and support throughout this assignment. Their detailed explanations about Java programming, GUI development, database integration, and software design enabled the successful completion of this project.

I am also thankful to Dr. Ilmini for her guidance on the database design module, which helped me to understand relational modelling and ER diagram construction more clearly.

My deepest appreciation goes to my family for their continuous support and encouragement, which enabled me to stay dedicated to my work throughout this assignment.

Table of Contents

1. Introduction	4
2. System Objectives	5
3. Requirements Analysis.....	5
3.1 Functional Requirements	5
3.2 Non-Functional Requirements.....	6
4. Graphical User Interface (GUI) Design	7
4.1 Screen Descriptions	11
5. Database Design.....	13
5.1 Database Choice	13
5.2 Entity Relationship Diagram (ERD).....	13
5.3 Table Definitions	14
6. Class diagram	17
7. Software Design – OOP Class Structure.....	18
7.1 Class Hierarchy.....	18
7.2 Class Descriptions	18
7.3 OOP Principles Applied	19
8. Testing.....	20
8.1 Test Plan	20
8.2 Test Cases	20
8.3 Sales Calculation Test Data.....	21
9. Challenges and Limitations.....	22
9.1 Challenges Encountered	22
9.2 Limitations.....	23
10. Conclusion.....	24
11. References	25

1. Introduction

The fast expansion of computer systems has changed how businesses conduct their daily operations through their retail operations which require precise work and effective processes and accurate record maintenance. The sales management system handles product data and customer information and sales processing and invoice creation while protecting data in a database which stores information for future access.

The project develops a sales management system which uses Java to create a graphical user interface based on object-oriented programming design principles. The system enables users to log in while they manage products and customers and process sales transactions and create invoices. The system uses JDBC to connect with a relational database which stores and retrieves permanent data about products and customers and sales records.

The system aims to replicate a small retail business operation through its graphical interface which enables shop assistants and administrators to complete daily sales tasks with high speed and perfect precision. The development team focused on implementing essential OOP principles which included encapsulation and inheritance and abstraction and modular design together with effective error management and validation procedures and structured testing.

The report contains all stages of system development which include analyzing requirements and designing the graphical user interface and modeling the database and constructing the class system and developing testing methods and solving problems and reaching the final results.

2. System Objectives

The primary objective is to develop a fully functional Java-based Sales Management System that enables users to:

- Log in to the system with valid credentials
- Manage products – add, update, delete, and view the product catalogue
- Manage customers – add, update, delete, and view customer records
- Create sales transactions by selecting customers and adding products to a cart
- Automatically calculate subtotals, discounts, and total amounts
- Generate and view invoices for completed transactions
- Export invoices as text files for record-keeping
- Store and retrieve all data persistently from a MySQL database via JDBC

3. Requirements Analysis

3.1 Functional Requirements

3.1.1 User Authentication

- Login screen with username and password fields
- Only authenticated users can access the system
- Invalid credentials produce a clear error message

3.1.2 Product Management

- Add new products: Product ID (auto-generated), Name, Unit Price, Quantity
- Update existing product details
- Delete products from the catalogue
- View all products in a sortable JTable

3.1.3 Customer Management

- Add customer details: Customer ID (auto-generated), Name, Phone Number
- Update and delete customer records
- View the full customer list in a JTable

3.1.4 Sales Transactions

- Select a customer for the transaction
- Select products and specify quantities
- Automatically calculate subtotal, optional discount, and total amount
- Save the completed sale transaction to the database atomically
- Stock levels are automatically reduced upon sale completion

3.1.5 Invoice Generation

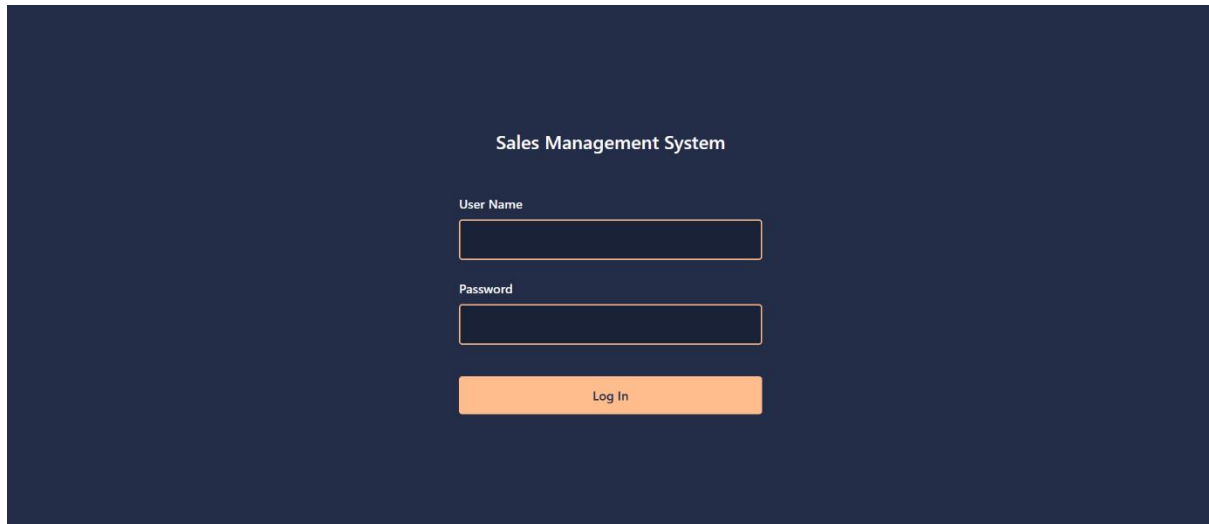
- Display invoice details on screen in the Invoice View screen
- View line items for each invoice
- Export invoice as a formatted text file

3.2 Non-Functional Requirements

- User-friendly GUI with consistent colour scheme and clear navigation
- Input validation: no negative prices or quantities permitted
- Proper error handling for database errors and invalid input
- Clean, readable, and well-commented source code
- Database transactions used to ensure data integrity

4. Graphical User Interface (GUI) Design

The system was developed using Java Swing. All screens follow a consistent dark-themed design (background color #2D3250) with orange accent buttons (color #F9B17A) for primary actions.



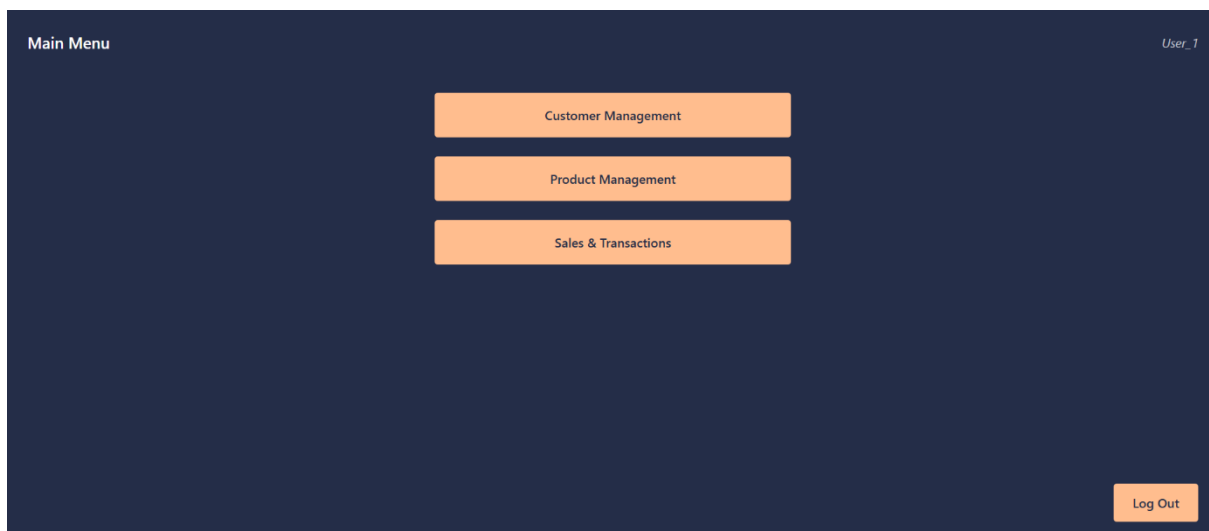
The login screen features a dark blue background with the title "Sales Management System" centered at the top. Below the title, there are two input fields: "User Name" and "Password", each with an orange border. A single orange "Log In" button is positioned below the password field.

Sales Management System

User Name

Password

Log In



The main menu screen has a dark blue background. In the top left corner, it says "Main Menu", and in the top right corner, it says "User_1". In the center, there are three stacked orange buttons labeled "Customer Management", "Product Management", and "Sales & Transactions". In the bottom right corner, there is an orange "Log Out" button.

Main Menu

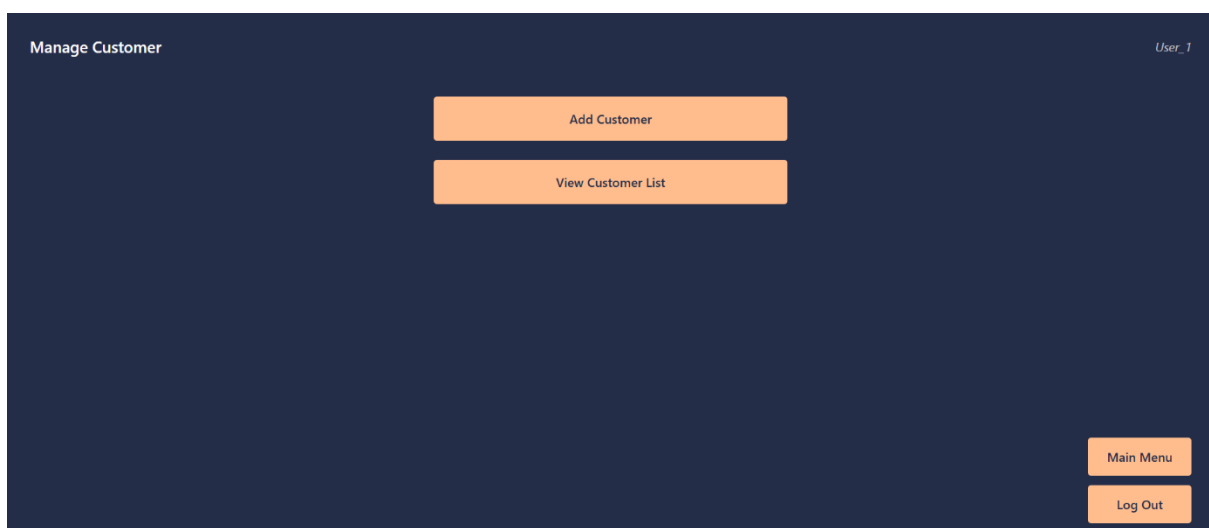
User_1

Customer Management

Product Management

Sales & Transactions

Log Out



The "Manage Customer" screen has a dark blue background. In the top left corner, it says "Manage Customer", and in the top right corner, it says "User_1". In the center, there are two stacked orange buttons labeled "Add Customer" and "View Customer List". In the bottom right corner, there are two stacked orange buttons labeled "Main Menu" and "Log Out".

Manage Customer

User_1

Add Customer

View Customer List

Main Menu

Log Out

Add Product

Update Product

Delete Product

View Products

Main Menu

Log Out

Name

Contact

Add

Main Menu

Back

Log Out

Name

Unit Price

Stock

Add

Main Menu

Back

Log Out

Delete Product

User_1

Product Id

Search

Product Name:

Unit Price:

Stock:

Delete

Back

Main Menu

Update Product

User_1

Product Id

Search

Product Name

Unit Price

Stock

Update

Back

Main Menu

Registered Customers

User_1

Customer Id	Name	Phone no.
C001	John Doe	0771234567
C002	Jane Smith	0772345678
C003	Mike Johnson	0773456789
C004	Sarah Williams	0774567890
C005	David Brown	0775678901
C006	Emily Davis	0776789012
C007	Robert Wilson	0777890123
C008	Lisa Anderson	0778901234
C009	James Taylor	0779012345
C010	Mary Thomas	0770123456

Main Menu

Back

Log Out

Product Id	Product Name	Unit Price	Stock
P001	Rice (1kg)	LKR 250.00	100
P002	Flour (1kg)	LKR 180.00	75
P003	Sugar (1kg)	LKR 220.00	50
P004	Tea (200g)	LKR 450.00	60
P005	Milk Powder (400g)	LKR 850.00	40
P006	Cooking Oil (1L)	LKR 520.00	35
P007	Bread	LKR 120.00	80
P008	Eggs (10 pack)	LKR 380.00	45
P009	Butter (500g)	LKR 780.00	25
P010	Yogurt (400g)	LKR 320.00	55

Back

Main Menu

Sales & Transactions

Phone

Search

Product

Select Product

Quantity

Add

Remove

Stock Reference

ID	Product	Stock
P001	Rice (1kg)	100
P002	Flour (1kg)	75
P003	Sugar (1kg)	50
P004	Tea (200g)	60

Transaction Cart

Product	Quantity	Unit Price	Total
No items in cart			

SubTotal: LKR 0.00

Save Sale

Invoice

Clear

Main Menu

Invoice

Customer : Niki

Phone : 0715555555

Date : 31 Mar 2026 19:51

Served by : User_1

Product	Qty	Unit Price (LKR)	Total (LKR)
Rice (1 kg)	1	185.00	185.00
Chocolate (100...	1	100.00	100.00
Milk Powder (4...	1	720.00	720.00

Total

LKR 1005.00

Thank you for your purchase!

Close

4.1 Screen Descriptions

Login Page

This is the entry point of the system. It has fields for the entry of a username and password, Login button, App logo/title. If there is an error logging in, a JOptionPane error dialogue will be shown with no indication which of the user name or password you have incorrectly entered.

Main Menu / Dashboard

When successfully logged in, the Main Menu will appear and display the name of the user that is currently logged in. Navigation buttons to the 4 management modules will be shown (Customer Management, Product Management, Sales Transaction, Invoice View). The Log Out button will return you to the Login Page.

Product Management Page

This page contains all CRUD operations for products. A JTable will be used to display the data (all products will be displayed). A user can click on any row in the JTable and the form fields will be pre-filled to allow for quick editing. The following buttons will be displayed, Add Product, Update, Delete, Clear, Refresh Table, Back. Input Validation will be done to ensure there are no negative numbers for price or quantity.

Customer Management Page

This page contains all CRUD operations for customers. Similar to the Lay Out for Product Management, Customer Management page will contain a JTable and Form Fields for Customer's Name and Phone Number. The logged in user's ID will automatically be recorded for the creation and updating of each customer. *Sales.*

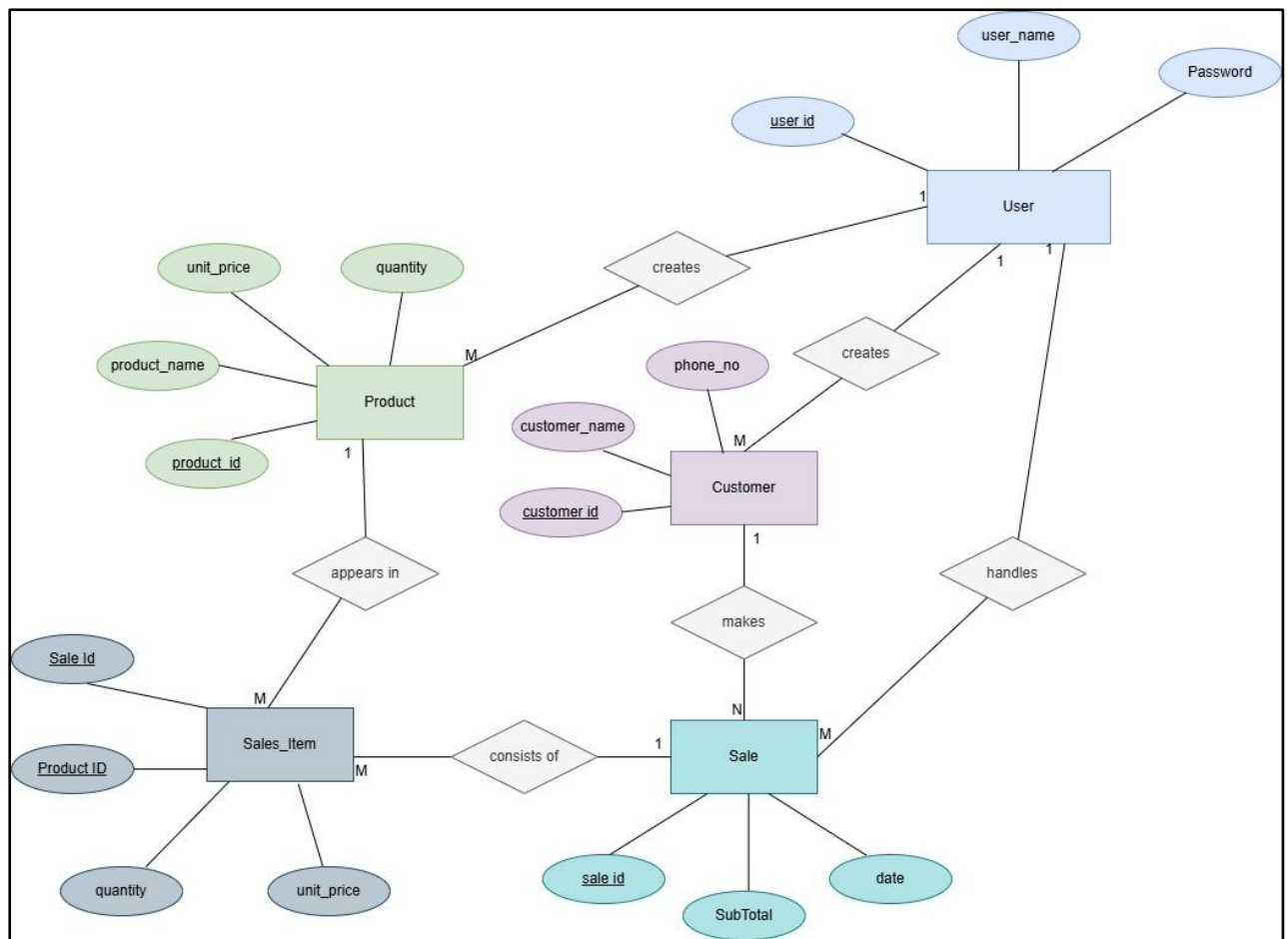
Transaction Screen

This interface enables users to create a transaction. The user can select the customer from a dropdown list. The products are selected from a dropdown and quantities can be entered before clicking the “Add to Cart” button to add them to the transaction. The transaction will display all the line items on the screen with an associated running total. If applicable, a discount can be applied to reduce the final amount of the transaction. The “Save Transaction” button saves the transaction to the database and removes the appropriate quantities from stock as necessary.

Invoice View Screen

The invoice view shows a table of completed transactions. By selecting a transaction from the list above, the items in the selected transaction will be displayed below in a second table. When the line items for the selected transaction are displayed, the invoice fields (customer, date, total) will be populated. An “Export Invoice” button allows exporting the selected invoice as a formatted .txt file

5. Database Design



5.1 Database Choice

MySQL was chosen as the database management system. The database is named sales_db and is accessed via JDBC using the MySQL Connector/J driver.

5.2 Entity Relationship Diagram (ERD)

The database models five entities with the following relationships:

- users → customer (one-to-many: a user creates/updates many customers)
- users → product (one-to-many: a user manages many products)
- users → sale (one-to-many: a user processes many sales)
- customer → sale (one-to-many: a customer has many sales)
- sale → sale_item (one-to-many: a sale contains many items)
- product → sale_item (one-to-many: a product appears in many sale items)

5.3 Table Definitions

Table: users

Column	Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
user_name	VARCHAR(25)	NOT NULL, UNIQUE	Login username
password	VARCHAR(255)	NOT NULL	Login password

Table: customer

Column	Type	Constraints	Description
customer_id	INT	PK, AUTO_INCREMENT	Unique customer identifier
customer_name	VARCHAR(255)	NOT NULL	Customer full name
phone_no	VARCHAR(20)	NOT NULL, UNIQUE	Contact number
created_by	INT	FK → users	User who added the record
created_date	DATETIME	DEFAULT NOW()	Record creation time

Table: product

Column	Type	Constraints	Description
product_id	INT	PK, AUTO_INCREMENT	Unique product identifier
product_name	VARCHAR(255)	NOT NULL	Product name
unit_price	DECIMAL(7,2)	NOT NULL, CHECK(>0)	Selling price per unit
quantity	INT	NOT NULL, CHECK(≥0)	Current stock level
updated_by	INT	FK → users	Last user to update
created_date	DATETIME	DEFAULT NOW()	Record creation time
updated_date	DATETIME	ON UPDATE NOW()	Last modification time
is_active	TINYINT	NOT NULL DEFAULT 1	Checks if the record is active (1) or inactive (0). All records are set as active by default.

Table: sale

Column	Type	Constraints	Description
sale_id	INT	PK, AUTO_INCREMENT	Unique sale identifier
customer_id	INT	FK → customer	Customer for this sale
user_id	INT	FK → users	User who processed the sale
sub_total	DECIMAL(9,2)	NOT NULL	Sum before discount
sale_date	DATETIME	DEFAULT NOW()	Date and time of sale

Table: sale_item

Column	Type	Constraints		Description
sale_id	INT	PK → sale,	COMPOSITE PK	sale
product_id	INT	PK → product		Product sold
quantity	INT	NOT NULL, CHECK(>0)		Quantity sold
unit_price	DECIMAL(7,2)	NOT NULL		Price at time of sale

6. Class diagram



7. Software Design – OOP Class Structure

7.1 Class Hierarchy

The system implements a class hierarchy using inheritance from an abstract base class:

Person (abstract) → User

Person (abstract) → Customer

Person is an abstract class holding the common name attribute, inherited by both User and Customer. This demonstrates the OOP principle of inheritance and abstraction.

7.2 Class Descriptions

Class	Type	Responsibility
Person	Abstract class	Base class with name attribute; parent of User and Customer
User	Child class	Extends Person; holds userId and password; represents authenticated users
Customer	Child class	Extends Person; holds customerId, phoneNo, timestamps
Product	Concrete class	Holds productId, productName, unitPrice, quantity
Sale	Concrete class	Holds saleId, customer, subTotal, date, user
SaleItem	Concrete class	Holds product, sale reference, quantity, unitPrice at time of sale
DatabaseConnection	Utility class	Provides static getConnection() factory method
LoginController	Controller class	Authenticates users against the users table
ProductDAO	DAO class	Full CRUD + stock reduction for products
CustomerDAO	DAO class	Full CRUD + list retrieval for customers

Class	Type	Responsibility
SaleDAO	DAO class	Saves sales atomically (header + items + stock) and retrieves invoices
LoginUi	View class	Login screen (Java Swing JFrame)
mainUi	View class	Main menu / dashboard
ProductManagementUI	View class	Product CRUD screen with JTable
CustomerManagementUI	View class	Customer CRUD screen with JTable
addCustomerUI	View class	Add/edit customer form dialog
viewCustomerUI	View class	Customer list view with search
SalesTransactionUI	View class	Cart-based sales transaction screen
InvoiceViewUI	View class	Invoice browser with export capability

7.3 OOP Principles Applied

Principle	How Applied
Encapsulation	All model class fields are private; access is through public getters and setters only
Inheritance	User and Customer extend the abstract Person class, reusing the name attribute and getName()/setName()
Abstraction	Person is declared abstract; DatabaseConnection provides a single abstract connection method; DAO classes hide all SQL from the UI layer
Polymorphism	toString() is overridden in Person, User, Customer, Product, Sale, and SaleItem to provide meaningful object representations
Modularity	Clear separation between Model, DAO, Controller, and View layers following an MVC-inspired pattern

8. Testing

8.1 Test Plan

Functional testing was performed manually by executing each use case against the live database.

Eight test cases were designed to cover authentication, product CRUD, customer management, sales calculations, and database operations.

8.2 Test Cases

TC#	Test Case	Input / Steps	Expected Result	Actual Result	Pass/Fail
TC01	Valid Login	Username: admin, Password: admin123	Dashboard opens; welcome message shows admin	Dashboard opened correctly	Pass
TC02	Invalid Login	Username: admin, Password: wrongpass	Error dialog: Invalid credentials	Error dialog displayed	Pass
TC03	Add Product	Name: Bread, Price: 120.00, Qty: 50	Product saved; appears in product table	Product added and displayed	Pass
TC04	Negative Price Blocked	Name: Tea, Price: -10, Qty: 20	Validation error: price cannot be negative	Error message shown; not saved	Pass
TC05	Update Product	Select product, change price to 150.00, click Update	Updated price reflected in table	Price updated correctly	Pass
TC06	Delete Product	Select product, click Delete, confirm Yes	Product removed from table and database	Product deleted	Pass
TC07	Create Sale Transaction	Select customer, add 2 products Save	Sale saved; total saved, stock reduced	Sale saved; stock reduced correctly	Pass
TC08	View Invoice	Open Invoice View after the sale is done	Invoice created with correct totals and items	Invoice UI viewed with correct data	Pass

8.3 Sales Calculation Test Data

The following calculation was verified manually for TC07:

Product	Qty	Unit Price (LKR)	Line Total (LKR)
Rice (1 kg)	2	185.00	370.00
Sugar (1 kg)	1	220.00	220.00
TOTAL			590.00

9. Challenges and Limitations

9.1 Challenges Encountered

- **Database Management:** It is important that when committing to the Database (DB), all parts (Sale Header, all Sale Items and reduction of Inventory) must occur as one commit. This required using `setAutoCommit(false)`, and implementing try and catch/rollback design patterns and managing connections correctly.
- **Database-UI Connection:** All screens containing the logged-in User object must be developed to allow for Constructor Injection. In addition, all DAO methods must also be designed so that the User ID will be available for use in all audit fields (`created_by`, `updated_by`).
- **JTable Synchronization:** The model for all JTable must be re-queried from the DB after each add/update/delete operation to ensure that the data displayed to the User is always accurate.
- **Foreign Key Constraints:** Initial testing of the application showed that attempting to delete from the Product table, any product which was previously in a sale items table failed due to foreign key constraint. To remedy, we used `ON DELETE CASCADE` for the Foreign Key between `sale_item` and `Sale` causing them to delete in one transaction and trained Users on how to not delete products with Sale history.
- **NetBeans Form Serialization:** NetBeans GUI Builder `.form` files cannot be edited outside of the IDE easily. Therefore, all new screens (`ProductManagementUI`, `SalesTransactionUI`, and `InvoiceViewUI`) were created in pure Java code without `.form` files to maintain portability.

9.2 Limitations

- **Plain Text Style Passwords:** Passwords are stored in an unsecured manner (plain text) and should be encrypted using a hashing algorithm such as BCrypt in any production environment.
- **No Role-Based Access Control Implementation:** Every authenticated user has the same level of access. Enhancing the system to provide an admin user with much more restrictive access to manage kind and user activities than the average user.
- **No Ability to Search/Filter Data:** No ability to search through database records for products or customers. Ability to search can be accomplished using DocumentListener applied to a search field widget.
- **Single Database Configuration Setting:** The database configuration settings (Connection URL, username and password) are currently hardcoded in the each DAO class. A production-ready solution would utilize a configuration properties file to hold this information.

10. Conclusion

This project was designed and implemented successfully using a Java desktop computer application based upon the Swing GUI component and MySQL Database as the backend. The project met all of the functional requirements set forth in the assignment brief: user authentication, Product Management (full CRUD), Customer Management (full CRUD), Sales Transaction Processing with automatic calculations and Invoice Generation.

This project illustrates the practical application of core Object-Oriented Programming principles such as encapsulation, inheritance and polymorphism; furthermore, the use of the Data Access Object (DAO) design pattern provided a clean separation between business logic and database access making the code base maintainable and extensible as well as ensuring database transactions provide data integrity when saving sales, preventing partially written records.

All eight of the identified test case(s) passed their respective tests; these included all of the following testing aspects: input validation, database operations and sales calculation accuracy.

This project has greatly increased my understanding regarding Java Swing GUI development, JDBC Database integration, DAO Design Patterns and the practical application of OOP Principles; all of which are directly applicable to me becoming a professional software developer.

11. References

- JDBC API Guide, 2023. *Oracle Corporation*. [Online]
Available at: <https://docs.oracle.com/javase/tutorial/jdbc/>
- MySQL 8.0 Reference Manual, 2023. *MySQL*. [Online]
Available at: <https://dev.mysql.com/doc/>
- NetBeans IDE Documentation, 2023. *Apache NetBeans*. [Online]
Available at: <https://netbeans.apache.org/>
- Oracle Corporation, 2023. *Oracle Corporation*. [Online]
Available at: <https://docs.oracle.com/javase/>